

Documento de visión - Valoración de mercado de Jugadores de fútbol

1. Descripción general

Se deberá realizar un desarrollo backend con una suite de APIs REST expuesta que integre información de jugadores de fútbol de 5 ligas principales: Premier League (Inglaterra), Bundesliga (Alemania), La Liga (España), Serie A (Italia) y Ligue 1 (Francia). El sistema deberá calcular una cotización periódica basada en criterios definidos y permitir a los usuarios operar comprando y vendiendo tokens. Asimismo, deberá gestionar el portfolio de cada usuario, mostrando su posición actual y rentabilidad.

El sistema deberá contemplar tanto procesos de lectura como operaciones transaccionales, así como la integración con APIs externas.

Fuentes de datos

- WhoScored: se utilizará scraping para extraer datos detallados de rendimiento de jugadores y equipos (pases, tiros, intercepciones, calificaciones, entre otros).
- Football-Data.org: se utilizará la API oficial para obtener resultados de partidos, alineaciones y fixtures.

Funcionalidades requeridas

- Definir la estructura de datos a utilizar
- Implementar un scraper de jugadores
- Simular una cotización de jugadores según estrategia
- Obtener la cotización actual de un jugador
- Obtener la cotización de un jugador para una fecha dada

2. Dominio del problema

El sistema representa un mercado de jugadores basado en criterios de valuación, donde:

- Cada jugador tiene una cotización que varía en el tiempo
- Los usuarios pueden invertir comprando tokens de jugadores
- El valor de la inversión cambia según la cotización actual

Ejemplo:

- Un jugador tiene una cotización de 100.
- Un usuario compra 10 tokens, invirtiendo 1000.

- Si la cotización sube a 120, su posición pasa a valer 1200, obteniendo una ganancia de 200.
- Si la cotización baja a 90, su posición pasa a valer 900, generando una pérdida de 100.

3. Funcionalidades obligatorias

3.1 Catálogo de jugadores

El sistema deberá:

- Integrar jugadores de las 5 ligas
- Obtener datos desde APIs externas
- Persistir la información localmente o BD cacheada

3.2 Sistema de cotización

El sistema deberá calcular una cotización periódica para cada jugador.

Requisitos

- La cotización debe recalcularse automáticamente (por ejemplo, semanalmente)
- Debe almacenarse el historial de cotizaciones
- Debe poder consultarse la cotización actual e histórica

<https://www.whoscored.com/players/234364/matchstatistics/ludovic-ajorque>

<https://www.whoscored.com/players/300713/matchstatistics/kylian-mbapp%C3%A9>

<https://www.whoscored.com/players/83532/matchstatistics/harry-kane>

Criterio de valuación

El sistema deberá implementar al menos **dos estrategias configurables de ponderación del valor de los jugadores**, basadas en distintos criterios de performance y contexto.

Cada estrategia deberá calcular un score del jugador a partir de un conjunto de métricas, tales como minutos jugados, goles, asistencias, tiros al arco, pases realizados, intercepciones, tarjetas amarillas y rojas, posición del jugador, rating general u otras métricas disponibles.

Las métricas podrán tener impacto positivo o negativo en el valor. Por ejemplo, sumar goles o asistencias incrementa el score, mientras que jugar menos de 90 minutos, cometer muchas faltas o recibir tarjetas amarillas o rojas puede reducirlo.

Cada métrica deberá tener un peso configurable dentro de la estrategia, permitiendo definir distintas formas de valuación.

El valor del jugador deberá derivarse de este score (por ejemplo, mediante una fórmula de ponderación o normalización).

Las estrategias deben poder seleccionarse, configurarse y dejar traza de cuál fue utilizada en cada cotización.

Ejemplo de estrategia basada en métricas de partido

Se propone una estrategia que calcule un **score de performance** a partir de métricas agregadas (por partido o ventana semanal) y lo convierta en valor.

Ejemplo (normalizando cada métrica en [0,1]):

- goals, assists, shots, keyPasses, dribbles, tackles, rating

Fórmula de score:

- $\text{score} = 0.25 * \text{goals} + 0.15 * \text{assists} + 0.10 * \text{shots} + 0.10 * \text{keyPasses} + 0.10 * \text{dribbles} + 0.10 * \text{tackles} + 0.20 * \text{rating}$

Conversión a precio:

- $\text{valor} = \text{valorBase} + (\text{score} * \text{factorEscala})$

Notas:

- Se puede ponderar distinto según **posición** (FW prioriza goals/shots, DF prioriza tackles).
- El cálculo se ejecuta en un **job semanal** y se guarda el histórico de cotizaciones.
- Debe registrarse la **versión de la estrategia** usada en cada cotización.

3.3 Mercado de tokens

Para simplificar el dominio, existirá un único superusuario que será el dueño inicial de todos los tokens de los jugadores. Cada jugador tendrá un total de 100 tokens emitidos. En el momento cero, cada token tendrá un valor inicial de 1 crédito. A partir de ese momento, la cotización de los tokens irá cambiando según la estrategia de valuación configurada en el sistema y la evolución del valor del jugador.

Los usuarios podrán comprar y vender tokens de jugadores. En una operación de compra, el sistema deberá validar que exista disponibilidad de tokens, utilizar la cotización vigente del jugador, actualizar la posición del usuario y registrar la operación. En una operación de venta, el sistema deberá validar que el usuario posea la cantidad de tokens a vender, actualizar su saldo, ajustar su posición y registrar la operación. Las operaciones de compra de los usuarios deberán realizarse inicialmente contra el superusuario, quien concentra la tenencia inicial de todos los tokens.

El sistema deberá permitir el registro y creación de nuevos usuarios para operar en el mercado.

3.4 Portfolio del usuario

El sistema deberá permitir visualizar la posición del usuario.

Debe incluir

- cantidad de tokens por jugador
- precio promedio de compra
- valor actual
- ganancia o pérdida
- historial de operaciones

4. Requerimientos funcionales (APIs)

Los siguientes endpoints deberán ser implementados y expuestos por el sistema:

Método	Endpoint	Descripción	Contexto
GET	/players	Listado de jugadores (con filtros por liga, equipo, posición)	Catálogo
GET	/players/:id	Detalle de un jugador	Catálogo
GET	/players/:id/quotes	Historial de cotizaciones de un jugador	Cotización
GET	/players/ranking	Ranking de jugadores según estrategia activa	Cotización
POST	/quotes/recalculate	Recalcular cotizaciones (job manual)	Cotización
POST	/orders/buy	Comprar tokens de un jugador	Mercado
POST	/orders/sell	Vender tokens de un jugador	Mercado
GET	/users/:id/portfolio	Portfolio del usuario	Mercado
GET	/users/:id/transactions	Historial de operaciones del usuario	Mercado

5. Requisitos no funcionales

5.1 Observabilidad

- Implementación de logs estructurados para facilitar el análisis.
- Uso de Correlation IDs para garantizar la trazabilidad de solicitudes entre servicios.
- Implementación de health checks y exposición de métricas clave como latencia y tasa de error.

5.2 Auditoría

Se requiere un registro inmutable de todas las transacciones financieras. Cada operación debe auditar:

- Identificación del autor de la acción.
- Detalle de los cambios realizados y marca de tiempo (fecha/hora).

- Estado anterior y posterior al cambio.

5.3 Performance

- Definición de objetivos de rendimiento mediante SLAs claros.
- Optimización de la base de datos mediante el uso estratégico de índices.
- Implementación de estrategias de caché para consultas frecuentes y mitigación de la latencia de APIs externas.

5.4 Componentes Técnicos

- **Scheduler:** Se requiere la implementación de un sistema de tareas programadas (job scheduler) para la ejecución automática de procesos batch, como el recálculo semanal de cotizaciones y la actualización de datos de jugadores.
- **Caché:** Se requiere la implementación obligatoria de una capa de caché (ej. Redis, in-memory) para optimizar las consultas frecuentes y mitigar la latencia/fallas de las APIs externas, permitiendo que el sistema funcione con datos locales si la fuente externa no está disponible.

6. Seguridad y Documentación

6.1 Seguridad

- Estricta validación de todas las entradas de datos (input validation).
- Manejo seguro de tokens de autenticación y autorización.

6.2 Documentación

Es obligatorio documentar todos los endpoints de la API utilizando el estándar OpenAPI (Swagger).

6. Arquitectura esperada

El sistema deberá estar organizado en capas:

- Controllers
- Services
- Repositories
- Adapters (APIs externas)

7. Integración con APIs externas

El sistema deberá consumir al menos una API externa de datos de fútbol.

El sistema debe tolerar fallas del proveedor externo y continuar funcionando con datos locales.

8. Escenarios de prueba

Durante la evaluación se deberá poder demostrar, como mínimo, lo siguiente:

1. Construcción de la base de datos de jugadores a partir de las fuentes externas definidas.
2. Obtención de la cotización de un jugador de cada liga en distintas fechas, mostrando la evolución de su valor.
3. Creación de 4 usuarios y compra de 5 jugadores al inicio del campeonato, para luego mostrar la valoración actual de sus posiciones a la fecha.

Se evaluará:

- correcto funcionamiento
- diseño de arquitectura
- manejo de errores
- buenas prácticas
- justificación del modelo de valuación

9. Entrega de Frontend

El sistema deberá contar con una interfaz de usuario (Frontend) que consuma las APIs expuestas por el backend. Esta interfaz deberá permitir:

- Visualización del catálogo de jugadores con filtros.
- Visualización de detalles y evolución de cotizaciones de un jugador.
- Visualización del ranking de jugadores.
- Operar en el mercado (compra y venta de tokens) autenticándose como usuario.
- Gestión y visualización del portfolio personal.

El frontend deberá ser una aplicación web responsiva, comunicándose con el backend a través de protocolos estándar (HTTP/REST). Se valorará la experiencia de usuario (UX) y la claridad en la presentación de los datos financieros.